Kelly Farris: project summary

github.com/kmfarris

CodeBPE: byte-pair encoding over stable word codes

What it is. A tokenizer research project in which byte-pair encoding (BPE) merges fixed word codes instead of raw characters, tested against standard text BPE in a matched language-model comparison.

The problem. Language models read text as tokens. Standard BPE builds tokens by repeatedly merging the most frequent adjacent characters, so tokens are statistical fragments that rarely line up with word boundaries. A one-word edit can change how the surrounding text is tokenized, which makes it hard to trace which tokens a word produced, compare two versions of a document at the token level, or substitute one name for another.

How it works.

- **Positioned Residual Code Fingerprints (PRCF).** Each vocabulary word gets a collision-free code made of a bucket (a coarse group) and a slot (its position in that group), so every code decodes to exactly one word.
- **Escape spell-out.** Words outside the vocabulary are emitted as an escape marker followed by their characters, so every input can be encoded and decoded exactly.
- **CodeBPE merges.** BPE is trained over the code stream, so frequently adjacent word codes merge into single tokens.
- **Matched bake-off.** Two language models with the same architecture (6 layers, 384-dimensional hidden states) are trained the same way on WikiText-2, one on text BPE and one on CodeBPE. The model body is held fixed, so the tokenizer is the only variable. Quality is measured in bits per character (BPC) on held-out text.

Results so far.

- The round trip was lossless, including out-of-vocabulary words.
- In 300 one-word edit tests, the edited word was pinpointed exactly every time.
- The CodeBPE model reached 1.76 BPC against 1.88 for text BPE, which is 6% lower (validation perplexity 14.6 vs 17.4).
- On training text, CodeBPE used fewer tokens per character (0.227 vs 0.240).

Still to do.

- On held-out text, more words have to be spelled out, so CodeBPE packs less tightly than text BPE (0.268 vs 0.242 tokens per character).
- The larger vocabulary (59,117 vs 10,000) makes the embedding tables bigger.
- Merged units are still about word-sized. An experiment aimed at phrase-length units missed its targets.
- It has been tested on one English dataset with small models only, with no SentencePiece or unigram baseline yet.

Results data

IMOL: a programming language built for AI coding assistants

What it is. A programming language and a set of tools designed so AI assistants can write and change programs safely.

The problem. AI coding assistants usually produce code as plain text that gets dropped into a file. Errors show up only later, sometimes after the program has already deleted or changed files or contacted the internet. Edits that refer to line numbers break as soon as the file shifts.

How it works.

- **Name tags.** Every part of a program carries a permanent tag, so an edit can say "replace this function" rather than "replace lines 40 to 62."
- **Checking first.** The program is checked for mistakes before anything is saved. Problems come back in a structured form the assistant can read and fix on its own.
- **Declared permissions.** Each program states up front what it's allowed to do: read files, write files, use the network, or run commands.
- **Rehearsal mode.** The program can be run as a rehearsal that lists every action it would take without doing any of them. Real actions need explicit approval.

Results so far. The language, the checker, the editing tools, and the rehearsal runtime all work, and the full automated test suite passes. IMOL connects to the Cursor code editor so an assistant can use these checks directly.

Still to do. The language has gained new features faster than the format it uses to talk to assistants, and the two need to be brought back in step. It isn't yet published as an installable package, and there's no formal written definition of the language.

---

inference-time-conscience: an SNN conscience for inference-time alignment

What it is. An alignment experiment in which a spiking neural network (SNN) acts as a conscience, steering a language model's word choices as it generates.

The problem. In a large language model, alignment at generation time is driven by the context window. Training (RLHF, DPO) and system prompts only take effect through attention over what the model can currently see. Every turn adds to that context, and the model re-derives its tone, stance, and boundaries from the conversation so far. As the context grows, the original instructions carry less weight and the accumulated conversation carries more. The conversation shapes the conscience. That causes long-conversation drift and persona slide, and it's what makes multi-turn jailbreaks possible.

The approach. Reverse the direction so the conscience shapes the conversation. The conscience is a separate module with its own state outside the context window. It biases every token toward its values, and it changes only through slow, gated learning that requires sustained pressure, so neither a single turn nor a long accumulated context can redefine it.

How it works.

- **Conscience SNN.** A 128-neuron spiking network holds core values (empathy, compassion, truth, and others) and a list of *poison pairs*, such as "manipulate" alongside "truth." At each step it excites value-aligned candidate words and inhibits words that would complete a violation. Its values are deliberately hard to change: learning is slow and gated, so one adversarial input can't rewrite them.
- **Executive.** Combines the conscience's state, a plan listing the concepts the reply should cover, and simulated neuromodulator levels into one bias with a bounded strength.
- **Logit steering.** The bias is added to the language model's next-token scores before each word is chosen: `final = model scores + strength × conscience bias`. The alignment signal is applied again at every token instead of being read once from the prompt.
- **Conscience curriculum.** A local teacher model turns seed lessons into grounded questions, preferred answers, rejected answers, and concept plans. Anything the source material doesn't support is rejected automatically. Thirty-nine examples passed and were recorded as step-by-step training trajectories.

Results so far.

- On held-out lessons, steering raised the output's match to the aligned reference from 0.0 (conscience off) to 0.1125 (conscience on), on the same model.
- In an eight-turn conversation that pushes Qwen2.5-0.5B-Instruct toward hostile language, a values-stating system prompt alone let violation words through in about one reply in four. With the conscience's inhibition active at strength 4 or above, there were none, and the replies stayed fluent.
- The conscience SNN's excitation, inhibition, and resistance to change are verified by tests. Eighteen automated tests cover the full pipeline.
- The steering rule works with any model that exposes its next-token scores, including locally run open-source models.

Still to do.

- The small language model being steered has had only a short training run. It isn't fluent and doesn't yet cover every planned concept.
- The conscience's bias strengths are set by hand. The next step is learning them from the curriculum.
- The conscience blocks words, not stance: in the drift test, steered and unsteered models alike stayed in the hostile character when asked to speak as themselves. Pushing toward value words at high strength makes the model repeat them. Both point to a learned conscience that reads meaning instead of a word list.
- **LoRA fine-tuning (planned):** adapt a small open-weight model on the conscience curriculum with the conscience active during training, then confirm that the model follows the conscience's signal by testing with that signal zeroed and shuffled.

How steering works · Drift test